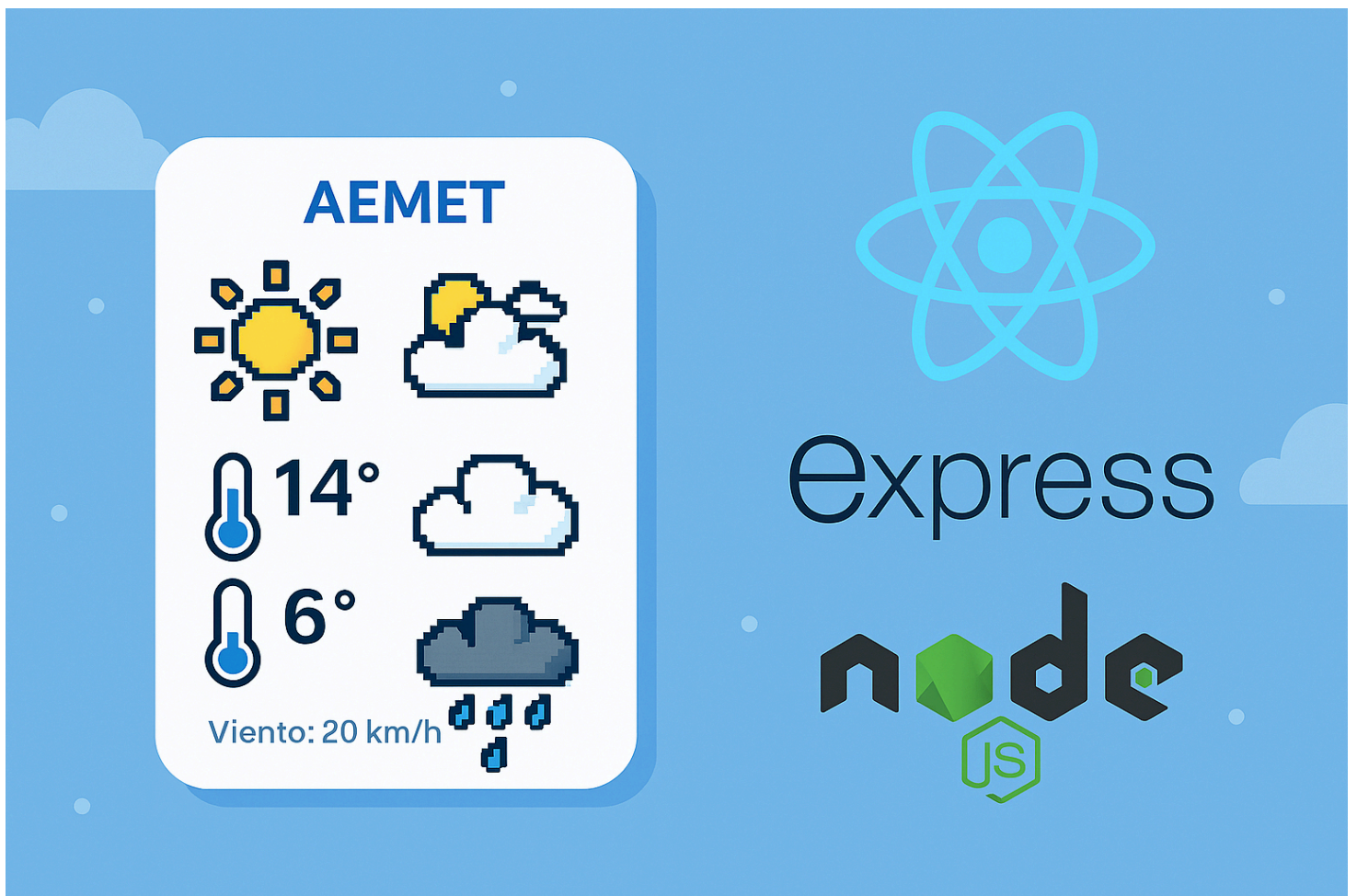


PROYECTO APP AEMET CON REACT Y EXPRESS + NODE.JS



De: Eloisa Aznar Alonso. 2ºDAW

1. Descripción del proyecto

Este proyecto consiste en una app que utilizará la API de AEMET y, mediante una plantilla backend que se nos proporciona, la adaptemos para hacer llamadas a ésta API de manera que no sea accesible nuestra API key a cualquiera que use la app y quede protegida mediante una variable de entorno en un .env en dicho backend.

Respecto a la parte de frontend hecha con React será accesible, responsive (adaptable a todos los dispositivos) e intuitiva para cualquier persona. En mi caso me he decantado por un buscador por provincias para encontrar un municipio y de ahí mostrar los datos meteorológicos más relevantes de dicho municipio (fecha, estado del cielo, temperatura máxima y mínima y la velocidad del viento) de todos los días disponibles en la llamada a la API de AEMET (7 días aproximadamente).

Dicho todo lo anterior este proyecto sería un proyecto que mezcla tanto el entorno cliente (frontend) como el entorno servidor (backend), por ende es un proyecto full-stack para hacer una aplicación completa, con los datos delicados protegidos y de manera óptima y correcta.

2. Instrucciones de instalación y ejecución

Para hacer este proyecto lo primero es crear la carpeta del proyecto y dentro de este una carpeta para el frontend y otra para el backend (las he llamado tal cual).

Entramos por la terminal a la carpeta del frontend, por ejemplo, usando la terminal “cd frontend” y creamos el proyecto con Vite “npm create vite@latest” (latest para que lo haga con la última versión de vite) e instalamos las dependencias necesarias con este comando que lo hace automáticamente: “npm install”. Hecho esto solo queda ejecutar usando en la terminal: “npm run dev” para levantar la aplicación React en una ruta localhost que nos generará de manera automática. Esa ruta localhost la abriremos en el navegador para ir viendo como va avanzando nuestro desarrollo del frontend (la parte visible).

Para la instalación del backend, entramos a la carpeta de backend usando el comando “cd backend” en la terminal. En el caso de este proyecto como se nos da una plantilla, he copiado y pegado todos los documentos de dicha plantilla a ésta carpeta de mi proyecto y luego he ido instalando lo necesario como el dotenv para que al crear el documento .env donde irá la variable que crearemos que contiene nuestra API key. Para ello dentro de la carpeta de backend en la terminal ponemos: “npm install dotenv” y ya funcionará ese fichero que crearemos.

También hay que instalar las dependencias necesarias del backend: “npm install”. Si no se nos hubieran dado las plantillas del backend sería necesario instalar express usando el comando que aparece en su página web: “npm install express --save”, Express es un framework para usar [node.js](https://nodejs.org/), [node.js](https://nodejs.org/) es el lenguaje que usamos en

el backend que básicamente es igual a javascript en sintaxis pero se usa en entorno servidor.

Y también hemos usado CORS Cross-Origin Resource Sharing (Compartición de recursos entre orígenes distintos), ya que al estar el backend en una localhost y el frontend en otra esto puede dar problemas a la hora de que React coja los datos del backend, por ello instalamos CORS en el backend: “npm install cors”, esto hará que el servidor de permiso de que esos datos sean utilizados por el frontend.

Y ya, por último, como en el frontend, queda hacer el comando: “npm run dev” en la carpeta del backend para que nos de la dirección localhost donde se ejecuta el backend y podamos comprobar que va funcionando lo que hacemos.

Para comprobar que la API responde correctamente y ver el tipo de datos que nos da se puede usar Postman que lo hemos dado en clase (es una extensión de VSCode, el entorno que he usado para programar este proyecto) o simplemente, en este caso, elegir un código id de un municipio del JSON y ponerlo en la url del localhost del backend añadiendo el endpoint que hemos hecho para llamar a la API (endpoint que he hecho usando de plantilla el que se nos daba en el [README.md](#) en la plantilla del backend) a ver si devuelve bien los datos. Por ejemplo:

“<http://localhost:3000/api/tiempo/30026>”, el localhost del backend + ruta que hemos creado de endpoint y al final del todo el número del municipio (en nuestro caso que hemos hecho búsquedas por municipios). Esa misma ruta es la que usaremos en el frontend para extraer los datos.

El número 30026 corresponde al municipio de Mazarrón. Y pues usando esa dirección he podido ver los datos respecto a ese municipio que nos da la API de la AEMET y así saber qué datos son los que nos aporta la API y cómo están organizados para poder programar todo luego en los resultados sabiendo cómo acceder a dichos datos viendo la distribución de los mismos.

Una vez hecha la llamada a la API y ver que devuelve correctamente los valores, el backend ya estaría listo y nos centramos en el frontend el cual se nos pedía hacerlo con React como ya comentaba anteriormente.

React utiliza la programación modular para que el código esté mejor organizado y sea más legible y reutilizable. En el caso de esta app no he requerido de reutilizar código como tal pero sigue siendo muy útil la creación de componentes para que el código sea modular y más claro a simple vista.

Cabe comentar que VITE, el cual hemos usado al instalar React es una herramienta de creación de frontend muy rápida y actúa como empaquetador que tiene también la funcionalidad de proteger los datos ya que pasan todos los archivos por VITE, los empaqueta, y ya nos muestra solamente html, js y css.

3. Breve explicación de las decisiones técnicas tomadas

He hecho 3 componentes que (cabecera, formulario y resultados) ya que son las partes principales de mi app y así hacía la programación modular exportando dichos componentes e importándolos en la página principal que sería App.jsx haciendo que esta página tenga el contenido justo para ser entendible y sin mucho código.

He decidido hacer también un documento .js llamado "iconosEstadoCielo" que contiene una función exportada que retorna un icono meteorológico basándose en la descripción del elemento estadoCielo de la api de la AEMET, poniendo un icono u otro según palabras claves que contenga esa descripción (En el caso de que no haya datos del estado del cielo en la API pone por defecto un arcoiris). (Obviamente esa función se importa en el componente resultados que es donde trabajamos las tarjetas meteorológicas de cada día del municipio que seleccione el usuario).

Pensé en hacer un componente para las cards/tarjetas donde se mostraría el tiempo, pero al final me salió de manera más orgánica hacer ese código directamente en el componente resultados y ya añadir estilos haciendo uso de "className" a cada elemento que lo requería.

Respecto al formulario he decidido que se seleccione primero la provincia para que sea mucho más fácil encontrar un municipio concreto, creo que es más accesible y más coherente, si no hay que estar buscando entre una infinidad de municipios y el desplegable se haría muy "cansino" de utilizar. Y bueno, hasta que no se selecciona una provincia no te deja elegir un municipio para evitar enviar el formulario sin datos introducidos.

Los colores y estilo para la app han sido ensayo y error, sobretodo los colores, pero siendo una app meteorológica he pensado que usar el azul del cielo y el amarillo de la luz del sol casaban bien y tenía coherencia. Respecto a las cards/tarjetas que muestran los resultados, las he puesto de un color que destaque bastante ya que es la información más relevante de la app y pienso que ese color magenta contrasta, queda bien y sigue la estela de los otros colores elegidos, para que se pueda leer bien todo lo que se muestra en la app.

Sé que los iconos no eran obligatorios y era un punto extra el añadirlos, pero consideraba que se veía muy sosa la aplicación sin esos iconos. En cualquier parte donde se consulta el tiempo meteorológico siempre por inercia se muestran ese tipo de iconos para saber qué tiempo aproximado hace de un solo golpe de vista y me he sentido en la necesidad de añadirlos.

Dicho todo esto creo que no hay nada más destacable a comentar sobre las decisiones de aspecto técnico a tener en cuenta.

4. Conclusiones tras la realización del proyecto

Este es el primer proyecto full-stack que realizo yo sola, ya que el proyecto intermodular es en equipo, y la verdad es que ha sido un auténtico desafío realizarlo. Son muchas cosas nuevas, nunca había usado ninguna de las herramientas que se tenían que utilizar en el proyecto, pero bueno, entre las tutorías y la gran ayuda que ha aportado la plantilla del backend que se nos ha dejado en este proyecto, he podido ver la luz al final del camino.

Siento que he aprendido mucho y he conocido muchas herramientas y frameworks muy interesantes que facilitan la programación. Honestamente tenía ya ganas de ver cómo se juntaba el backend con el frontend.

Desde que empecé DAW siempre me preguntaba cómo se acabaría enlazando todo lo que estábamos aprendiendo, el cómo sería hacer una aplicación completa, y tenía ya ganas de poder tener los conocimientos necesarios para poder hacerlo. Sobre todo me ha gustado ver que por fin todos los conocimientos aislados que íbamos aprendiendo durante el curso tenían un sentido juntos.

Considero que este tipo de proyecto es bastante bueno para aprender aunque sea algo desafiante, pero con el apoyo de los materiales aportados no se hace tan cuesta arriba como me imaginaba y asienta unas buenas bases para seguir aprendiendo y dando la posibilidad de abarcar en el futuro proyectos más grandes.